

Хакатон Московского транспорта

ИИ-прогноз загрузки трамвайных маршрутов

Краткое описание

В Едином диспетчерском центре (ЕДЦ) ГУП «Московский метрополитен» ведётся работа по созданию набора моделей краткосрочного и долгосрочного прогнозирования пассажиропотока на наземном транспорте. Участникам предстоит разработать математическую модель на базе технологий ИИ и ML, а также интерактивный веб-сервис для анализа больших данных телематики и валидаций (оплаты проезда). Решение должно позволять диспетчерам эффективно распределять подвижной состав, снижать переполнение салонов и оптимизировать затраты на эксплуатацию.

Проблематика

В настоящее время прогнозы пассажиропотока строятся на основе простейших регрессионных моделей, имеющих ограниченную точность. Телематика сама по себе не является исчерпывающим источником для прогнозирования. Отсутствует единое, масштабируемое решение, которое:

- Качественно агрегирует разрозненные массивы больших данных (валидации, расписания, геопривязка).
- Обеспечивает прогнозирование на разных горизонтах планирования с высокой точностью.
- Предоставляет удобный инструмент визуализации и выгрузки данных для принятия оперативных и стратегических решений.

Цель и целевые задачи

Цель: Разработать программное решение в виде веб-сервиса с пользовательским интерфейсом, использующее технологии ИИ/ML для прогнозирования пассажиропотока на маршрутах трамваев с привязкой к геопозиции и времени.

Целевые задачи:

1. Обеспечить получение прогноза на три горизонта: краткосрочный (день, с разбиением по часам на предикты), среднесрочный (месяц), долгосрочный (год - опциональный сценарий).
2. Реализовать агрегацию прогноза по заданным параметрам: маршрут, остановка/участок, интервал времени.
3. Создать пайплайн приёма, нормализации и геопривязки исторических данных валидаций по расписанию и остановкам.

Образ решения

Контейнеризированный программный продукт (веб-сервис: Backend + Frontend), развертываемый через Docker. Сервис предоставляет REST/HTTP API для получения прогнозов, интерактивный дашборд с визуализацией динамики загрузки на карте Москвы в реальном времени, а также функционал экспорта данных в табличном виде.

Функциональные требования

- Гибкое прогнозирование: Построение предиктивных моделей на горизонты 1 день, 1 месяц, 1 год.
- Детализированная агрегация: Фильтрация и агрегация прогноза по конкретным маршрутам, отдельным остановкам и выбранным временным интервалам.
- Интерактивный дашборд: Визуализация динамики изменения пассажиропотока по времени по маршруту в заданных точках (остановки) на карте Москвы.
- Экспорт данных: Возможность выгрузки результатов прогноза в форматах CSV и/или XLSX.
- Учет внешних факторов: Возможность учета времени суток, дня недели, сезонности и погодных условий (при наличии соответствующих данных).

Источники данных и ограничения

- Все данные предоставляются в виде zip-архива dataset.zip, содержащий детальное описание датасет, его полей, а также необходимые для разработки данные. - <https://disk.yandex.ru/d/DiFwlfMOauxjBg>.

Требования к архитектуре и рекомендации по технологическому стеку

Рекомендуемый прикладной стек:

- Backend: Java 17+ (рекомендуется Java 21 LTS), Spring Boot 4.0+ (рекомендуется 4.1.x), Netty (реактивный стек / Spring WebFlux для высокой пропускной способности).
- Frontend: React 19+, TypeScript 5.x+.
- ML-контур: Python (scikit-learn / LightGBM / PyTorch) с поставкой артефактов модели и/или инференс в JVM через ONNX Runtime / DJL.
- Инфраструктура: Docker, Docker Compose.

- Рекомендуемые компоненты: СУБД / хранилище истории (PostgreSQL, ClickHouse или файловое columnar-хранилище), библиотека карт для UI (MapLibre / Leaflet / Yandex Maps API).

Нефункциональные требования

- Производительность: Высокая пропускная способность сервиса под нагрузкой (приём потоковых данных и отдача прогнозов). Сервис — один контейнер 2–4 vCPU / 2–4 ГБ RAM, держащий сотни RPS при p95 < 200–300 мс и утилизации CPU 60–80% с запасом, RAM стабильна без swap. – В ОБЯЗАТЕЛЬНОМ ПОРЯДКЕ УКАЖИТЕ ДАННЫЕ О ПРОИЗВОДИТЕЛЬНОСТИ ВАШЕГО РЕШЕНИЯ В РИДМИ!
- Масштабируемость: Совместимость с горизонтальным масштабированием.
- Эффективность: Оптимальное использование ресурсов (CPU, RAM) в рамках одного экземпляра контейнера.
- Надёжность: Корректная обработка ошибок API, наличие понятных сообщений пользователю.
- Расширение за счет внешних данных - прогнозов погоды, транспортной ситуации, данных о строительстве и открытии новых маршрутов.

Рекомендуемые роли в команде

- 1-2 ML Engineer / Data Scientist: Разработка, обучение и тюнинг прогностических моделей временных рядов.
- 1 Backend-разработчик: Интеграция модели, создание стабильного и высоконагруженного API (Java/Spring).
- 1 Frontend-разработчик: Разработка визуального интерфейса, дашборда и визуализация данных на картах (React/TS).
- 1 Системный/Бизнес-аналитик или Product Manager: Проектирование пайплайнов данных, контроль соответствия требованиям, подготовка документации.

Критерии оценивания основного этапа

№	Критерий	Баллы
1	Качество прогноза (метрика WAPE-score)	0–10
2	Данные, внешние источники и область применимости модели	0–8
3	Архитектура и производительность	0–5
4	Полнота функциональности (горизонты, агрегация, карта, выгрузка)	0–4
5	Бизнес-ценность и применимость решения	0–2

Детализация критериев

Критерий 1. Качество прогноза (метрика WAPE-score) (0–5 с весом 2)

$WAPE\text{-score} = \max(0, 1 - \sum |y - \hat{y}| / \sum y)$ in $[0, 1]$, больше — лучше

Балл берётся по таблице соответствия (baseline ≈ 0.48 — «пол», который нужно превзойти):

WAPE-score	Баллы	С учетом веса (2)
≤ 0.48 (не лучше baseline)	0	0
0.48 – 0.60	1	2
0.60 – 0.70	2	4
0.70 – 0.80	3	6
0.80 – 0.88	4	8
> 0.88	5	10

Критерий 2. Данные, внешние источники и область применимости модели (0–8)

Складывается из трёх частей:

а) Внешние источники (до 4 баллов) — по 1 баллу за использованный и подтверждённый эффектом источник:

- **дорожный трафик / загруженность улиц;**
- **погодные условия** (температура, осадки и т.п.);
- **сезонность и календарь** (выходные, праздники РФ/Москвы, каникулы, сезонное переключение спроса);
- **прочие внешние факторы** (события, новости, перекрытия, ремонты и т.п.).

Обязательное условие: на каждый источник дана рабочая ссылка/способ получения. Источник без ссылки — не засчитывается.

б) Область определения и адаптации модели (до 2 баллов) — команда чётко понимает, на каких маршрутах/горизонтах/диапазонах данных модель валидна (область определения) и как/от каких внешних данных зависит, что нужно для переноса на новые маршруты и периоды (область адаптации). Явно описаны пределы применимости.

в) Корректирующие коэффициенты (до 2 баллов) — в интерфейсе есть возможность менять корректирующие коэффициенты (поправка на погоду/событие/сезон) и сразу видеть, как меняется прогноз (+1); базовая корректная агрегация целевой величины и воспроизводимый пайплайн приёма данных (+1).

Критерий 3. Архитектура и производительность (0–5)

- Соответствие общим архитектурным принципам построения решений.
- Читаемое разделение на модули: приём/нормализация данных → признаки/геопривязка → ML-прогноз/агрегация → API → frontend.
- REST/HTTP API отдаёт прогноз по параметрам (маршрут, остановка/участок, интервал, горизонт).
- Производительность и масштабируемость: **по заявленным командой метрикам** (latency/RPS в README) — отдельный нагрузочный тест жюри не проводит; честная оценка с подтверждением засчитывается.
- При отсутствии запускаемого сервиса балл существенно снижается.

Критерий 4. Полнота функциональности (0–4)

- Три горизонта прогноза: **день, месяц, год** (день обязателен; месяц и год — хотя бы качественно).
- Детализированная агрегация: по маршруту, остановке/участку, интервалу времени.
- Интерактивный **дашборд с картой Москвы**: динамика пассажиропотока по времени и точкам маршрута.
- **Выгрузка** результатов прогноза в CSV и/или XLSX.

Критерий 5. Бизнес-ценность и применимость (0–2)

- Ясно показано применение: распределение подвижного состава, снижение переполнения салонов, уточнение расписания, оптимизация эксплуатации.
- Реалистичность выводов из прогноза и понимание ограничений решения.

Загрузка решения

Решение загружается на платформе хакатона в двух обязательных версиях:

1. Загрузка csv-файла в разделе **«Data Science»**.

Лимиты загрузок на каждую команду в день:

- **общее кол-во попыток – 36** (все попытки, включая ошибочные);
- **кол-во успешных попыток – 24** (попытки со скором 0 и выше).

Загружать решение в данном разделе можно **до 27 сентября, 23:59 МСК**.

В зачет пойдет лучший результат, полученный за всё время.

2. Заполнение полей формы (ссылка/ввод текста) в разделе **«Загрузка решения»**.

Обновлять решение в данном разделе можно вплоть до дедлайна регистрации – **27 сентября, 23:59 МСК**.

Сохранится и пойдет на проверку **заключительная версия**.

Обязательные артефакты на сдачу

(заполнение полей формы на платформе хакатона с указанием ссылок на материалы)

1. Ссылка на артефакты ML-модели и код обучения/инференса, README с инструкцией запуска.
2. Ссылка на все внешние данные, использованные в решении (трафик, погода, календарь, новости и др.).
3. Ссылка на запускаемый веб-сервис (Docker/Docker Compose): backend + frontend, точки входа API, инструкция для жюри.
4. Ссылка на схему архитектуры и модулей; описание области определения и адаптации модели и зависимостей от внешних данных.
5. Сведения о производительности (замеры/нагрузочный тест) и список реализованных дополнительных возможностей.
6. Описание ограничений решения и плана развития после хакатона.

Оценка полуфинальных питчей

№	Критерий	Баллы
1	Оценка полуфинальных питчей	0–10

Итоговый максимум по этапам 1-2 — 39 баллов.

- Глубина погружения в тему и уровень проработки (0–4)
- Корректность и аргументированность ответов на вопросы жюри (0–3)
- Качество выступления и демо (0–3)